
Universidad Aeronáutica de Querétaro

Embedded Systems Laboratory

SCADE One Laboratory Series

Practice #9

Integration Project: The Flight Control System, RTOS-Native

Elaborated by: Leonardo Franco García

Student ID: 12148

Date: July 31, 2026

Full name: _____

Student ID: _____

Date: _____

1 Learning Objectives

Learning Objectives

By the end of this practice, the student will be able to:

- ✓ **Integrate a complete model-based system:** generate and run the full flight-control model — flight phase, four control surfaces, and built-in emergency logic — inside the RTOS architecture developed across Practices 5–8.
- ✓ **Drive multiple actuators in parallel:** fan the model's outputs out to four surface servos plus the landing-gear servo on independent LEDC channels, all from one actuator task.
- ✓ **Wire sensing, control, actuation, safety, and telemetry as cooperating tasks:** assign priorities so the control law holds its cycle while I/O and telemetry run around it.
- ✓ **Defend the design:** explain, subsystem by subsystem, what lives in the SCADE model versus the glue, and why the result is a controller that could be reviewed for certification rather than a working demo.

2 Introduction

This is the destination the whole course has aimed at. Every earlier practice added one layer — code generation, telemetry, tasks, state machines, sensors and actuators, functional safety — around an unchanging principle: the model owns the logic, the platform owns the I/O. Here those layers close into a single system: a flight-control computer that reads two pilot inputs, runs a SCADE model of the aircraft's flight phases and control-surface laws with its own emergency handling, drives four surface servos and the landing gear, indicates state on the panel LEDs, and streams telemetry — all as a priority-scheduled set of FreeRTOS tasks. The same behaviour may already exist as a single Arduino `loop()`; the work of this practice is to re-engineer it into the concurrent, fail-safe, model-based architecture that separates a prototype from a system you could certify.

3 Bill of Materials

#	Component / Software	Qty	Notes
1	SCADE One (Ansys)	1	Full flight-control model
2	ESP32 DevKit V1	1	Environment from Practice 2/3
3	USB-A to Micro-USB cable	1	Data cable (console + telemetry)
4	ESP-IDF (v5.x)	1	FreeRTOS + ADC/LEDC/WDT built in
5	Generated <code>scade_gen</code>	1	The complete model component
6	SG90 servos	5	4 surfaces + landing gear
7	Potentiometer 10 k Ω	2	<code>mode</code> (GPIO34), <code>dir</code> (GPIO35)
8	LEDs (red, green, white)	3	Panel: alarm / ok / status
9	220 Ω resistors	3	LED current limit
10	External 5 V supply (≥ 2 A)	1	<i>Five</i> servos — a strong supply
11	Breadboard + jumper wires	–	Shared 5 V and ground rails

Power is now the hard part

Five SG90 servos moving together can draw well over an ampere in peaks. Use a 5 V supply rated for at least 2 A, distribute it on a solid breadboard rail, and tie that rail's ground to the ESP32 ground. Under-powering the servos is the single most common failure of this practice — the board browns out or the servos twitch. Keep the pot ends on 3V3/GND (they draw almost nothing).

Signal	Role	Pin
<code>mode</code>	flight-phase pot (ADC1)	GPIO 34
<code>dir</code>	direction pot (ADC1)	GPIO 35
<code>aleizq</code>	left aileron servo (CH0)	GPIO 13
<code>aleder</code>	right aileron servo (CH1)	GPIO 12
<code>tdir</code>	rudder servo (CH2)	GPIO 14
<code>elev</code>	elevator servo (CH3)	GPIO 27
<code>gear</code>	landing-gear servo (CH4)	GPIO 4
red / green / white	panel LEDs	GPIO 25 / 26 / 33

4 Theory & Block Reference

Nothing in this practice is a new peripheral or a new API — every piece was built earlier. What is new is the *system*: how the subsystems compose into one priority-scheduled application around a single generated model. The reference below is that composition, not another driver.

4.1. The model is the aircraft; the glue is the airframe

The generated model takes the two pilot inputs (`mode`, `dir`) and produces everything that matters: the active flight phase, the four surface angles, the landing-gear intent, the panel indications, and — critically — its own `emergency` output, raised when it detects a lost

or jumping input. All of that is verifiable in the SCADE simulator before a single wire is connected. The glue does only what a model cannot: read the ADCs, drive the LEDC channels, blink an LED, kick the watchdog, frame telemetry. This is the same division from Practice 3, now spanning a whole aircraft.

Subsystem: Task architecture (the whole system)

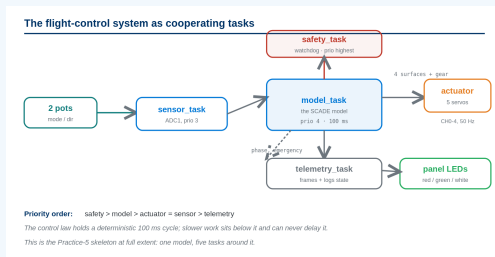


Figure 1: Sensor, control, actuator, safety and telemetry tasks around one model — annotated view.

Tasks, highest priority first:

- **safety:** kicks the watchdog and guards liveness (the model raises its own emergency).
- **model:** reads `mode/dir`, steps the model each 100 ms, distributes its outputs.
- **actuator:** writes the five servo channels from a queued command.
- **sensor:** samples the two pots.
- **telemetry:** frames and logs the aircraft state.

Behavior:

The control task holds a deterministic 100 ms cycle; slower or blocking work (telemetry, servo writes) sits below it and can never delay it. This is the Practice 5 skeleton at full extent.

Subsystem: Model interface (confirm in your header)

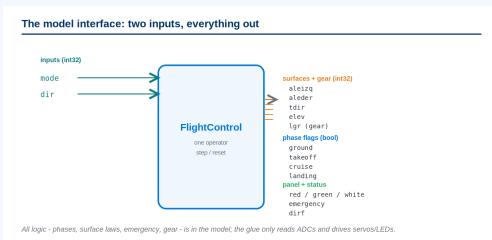


Figure 2: The generated step function: two inputs, surface / LED / emergency outputs — annotated view.

Inputs: mode, dir (raw ADC counts).

Outputs (typical): the four surface angles (aleizq, aleder, tdir, elev), the phase flags (tierra, despegue, cruise, aterrizaje), the panel LEDs (red, green, blanca), and emergency / safe.

Behavior:

Names and order are set by *your* model — open the generated header and match the glue to it exactly, as in every practice since 3. The gear angle is derived in the glue from the phase flags (cruise → up, ground/landing → down).

4.2. Where each requirement is met

Requirement	How it is met	From
Flight phases	state machine in the model	P6
Surface control law	dataflow in the model	model
Read pilot inputs	ADC in <code>sensor_task</code>	P7
Move surfaces + gear	LEDC in <code>actuator_task</code>	P7
Emergency reaction	<code>emergency</code> output of the model	P8
Liveness / hang	Task Watchdog in <code>safety_task</code>	P8
State reporting	framed <code>telemetry_task</code>	P4
Concurrency + timing	task priorities and queues	P5

5 Building the Flight-Control Model

This is the one new modeling effort of the practice. Every skill needed was taught earlier — dataflow (Practice 1/3), state machines (Practice 6), and an emergency state (Practice 8) — and here they combine into a single operator, `FlightControl`, with two inputs (`mode`, `dir`) and the outputs the glue expects. Build it in six layers, simulating after each; do not generate code until the whole model simulates cleanly. All names below are the model's; match the glue to the generated header exactly, as always.

5.1. Layer 1 — Centre the direction input (dataflow)

The direction stick reads 0–4095 with centre near 2048. Produce a signed deflection:

$$\text{dirf} = \text{dir} - 2048 \quad (\text{range } -2048 \dots +2047)$$

This single subtraction feeds every direction decision downstream.

5.2. Layer 2 — Flight phase (automaton on mode)

Model the phase as an automaton (Practice 6) whose transitions are thresholds on `mode`, splitting 0–4095 into four quadrants. Each state emits its own phase flag `true` and the others `false`.

state	active when	emits
GROUND	$\text{mode} \leq 1023$	ground=true
TAKEOFF	$1024 \leq \text{mode} \leq 2047$	takeoff=true
CRUISE	$2048 \leq \text{mode} \leq 3071$	cruise=true
LANDING	$\text{mode} \geq 3072$	landing=true

Add one more state, **EMERGENCY** (Layer 5), reachable from every phase on `emergency` — the guard takes priority over the mode transitions, exactly as in Practice 8.

5.3. Layer 3 — Surface control law (phase $\times$ direction)

Each flight phase deflects the surfaces differently, and within a phase the deflection depends on the direction. Model this as a second automaton in which every phase has three sub-states selected by `dirf`: *left* ($\text{dirf} < -300$), *centre* ($-300 \leq \text{dirf} \leq 300$), and *right* ($\text{dirf} > 300$). Each sub-state emits four constant angles for `ail_left`, `ail_right`, `rudder`, `elevator` (0–180, neutral = 90). A representative set:

phase	dir	ail_left	ail_right	rudder	elevator
any	centre	90	90	90	phase-trim
cruise	left	115	65	70	90
cruise	right	65	115	110	90

Ailerons move oppositely (roll), the rudder follows the turn, and the elevator holds a phase-dependent trim (higher on takeoff and landing). On **emergency**, an overriding transition forces all four surfaces to 90° (neutral).

5.4. Layer 4 — Emergency detection (dataflow)

The model raises **emergency** when an input is untrustworthy, by two independent tests OR-ed together:

- **Out of range:** $\text{mode} < 20$ or $\text{mode} > 4075$ — a railed reading means the wire is disconnected.
- **Sudden jump:** $|\text{mode} - \text{pre}(\text{mode})| \geq 2000$ — a one-cycle leap this large is not a human turning a knob.

The `pre` operator (Practice 1) gives last cycle’s value for the jump test. This is detection *in the model* — the alternative to the glue-side monitor of Practice 8; discuss the trade-off in the questionnaire.

5.5. Layer 5 — The EMERGENCY state and its latch

Route **emergency** into the two automata: from every phase and every surface sub-state, an **emergency**-guarded transition (highest priority) enters **EMERGENCY**, which sets all phase flags false and all surfaces to 90°. To avoid chatter on a momentary glitch, hold the emergency for a minimum number of cycles with a **pre**-based counter (**cont**) — the same debounce idea as Practice 8's persistence, now inside the model — and expose a **safe** flag that is true only once the counter has elapsed and the fault has cleared.

5.6. Layer 6 — Panel LEDs and the landing gear

A small LED automaton drives the panel: on the ground the green LED is on; in the air the white LED is on; in emergency the red LED *flashes*, toggled by the **cont** counter (states **EMERGENCY_ON** / **EMERGENCY_OFF**). Finally add the landing-gear output as an **int32** angle emitted directly by the phase automaton:

$$\text{gear} = \begin{cases} 90^\circ & \text{in GROUND, TAKEOFF, LANDING (gear down)} \\ 0^\circ & \text{in CRUISE and EMERGENCY (gear up / safe)} \end{cases}$$

so the model — not the glue — decides the gear position: down on the ground, during takeoff, and during landing, and up in cruise. In **EMERGENCY** it commands 0°. The glue simply forwards **gear** to its servo channel. Simulate the whole operator — sweep **mode** and **dir**, then force an out-of-range **mode** — and confirm the phases, surfaces, LEDs, gear, and emergency all behave before generating code.

Name it deliberately

The generated function and context names come from your operator and module names (Practice 3). Whatever you name this operator, open the generated header afterward and match every argument in the glue to it — the listing in the Main Activity uses representative names only.

6 Worked Example

6.1. Objective

Bring up all five servos before adding the model — the riskiest hardware step. Initialize one 50 Hz LEDC timer with five channels and sweep every servo to centre (90°), confirming each responds and the supply holds its voltage. About 30 minutes.

6.2. Step-by-Step

1. Create the project.

Copy the Practice 8 project (it carries the drivers and workflow):

```
cd C:\esp
xcopy /e /i lg_safe lg_aircraft
cd lg_aircraft
idf.py fullclean
```


2. Wire the five servos and measure power.

Signals to GPIO 13, 12, 14, 27, 4; all servo + to the 5 V rail, all - to the common ground tied to the ESP32. With the multimeter on the rail, confirm it holds 5 V while the servos move.

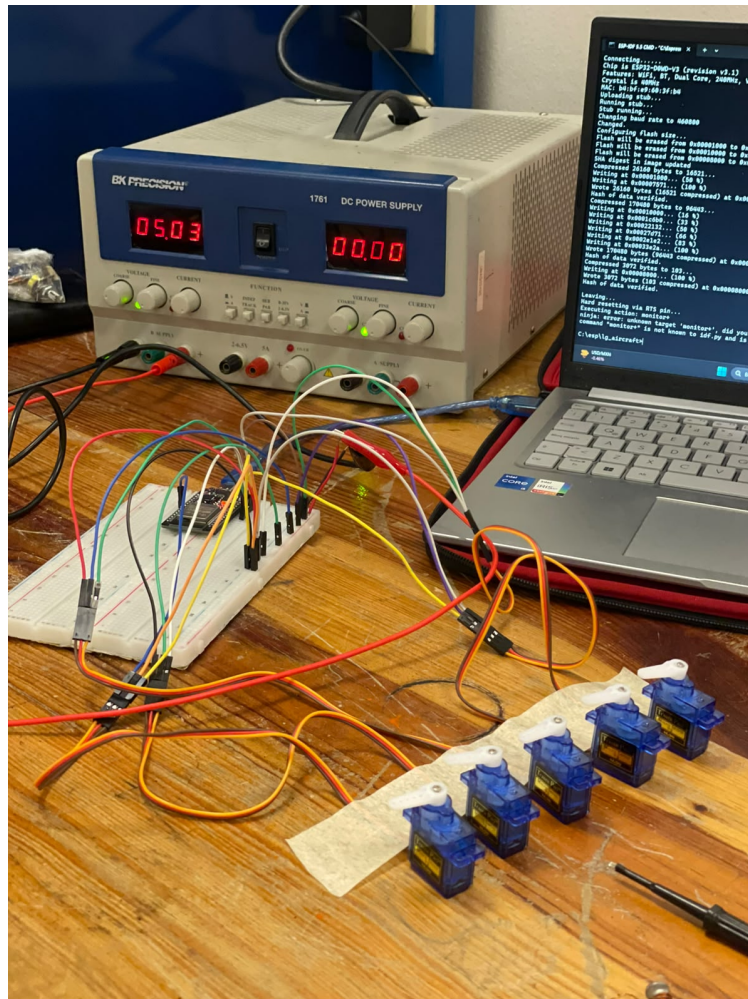


Figure 3: Five servos on a shared, well-powered 5 V rail.

3. Write the multi-servo bring-up.

Replace main.c:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/ledc.h"
#include "esp_log.h"
static const char *TAG = "AC";

static const int SERVO_GPIO[5] = { 13, 12, 14, 27, 4 };

static void servos_init(void)
{
    ledc_timer_config_t t = { .speed_mode = LEDC_LOW_SPEED_MODE,
        .timer_num = LEDC_TIMER_0, .duty_resolution = LEDC_TIMER_16_BIT,
        .freq_hz = 50, .clk_cfg = LEDC_AUTO_CLK };
    ledc_timer_config(&t);
    for (int i = 0; i < 5; i++) {
        ledc_channel_config_t c = { .gpio_num = SERVO_GPIO[i],
            .speed_mode = LEDC_LOW_SPEED_MODE, .channel = i,
            .timer_sel = LEDC_TIMER_0, .duty = 0, .hpoint = 0 };
    }
```



```
        ledc_channel_config(&c);
    }
}
static void servo_write(int ch, int deg)
{
    if (deg < 0)    deg = 0;
    if (deg > 180) deg = 180;
    int us = 500 + (2500 - 500) * deg / 180;
    uint32_t d = (uint32_t)((uint64_t)us * 65535 / 20000);
    ledc_set_duty(LEDC_LOW_SPEED_MODE, ch, d);
    ledc_update_duty(LEDC_LOW_SPEED_MODE, ch);
}

void app_main(void)
{
    servos_init();
    int deg = 0, step = 15;
    while (1) {
        for (int ch = 0; ch < 5; ch++) servo_write(ch, deg);
        ESP_LOGI(TAG, "all servos -> %d deg", deg);
        deg += step;
        if (deg >= 180 || deg <= 0) step = -step;
        vTaskDelay(pdMS_TO_TICKS(300));
    }
}
```

4. Declare dependencies, flash, and verify.

REQUIRES `scade_gen esp_driver_gpio esp_driver_ledc esp_adc`. Flash: all five servos sweep together. If any twitches or the board resets, the supply is too weak — fix power before continuing.

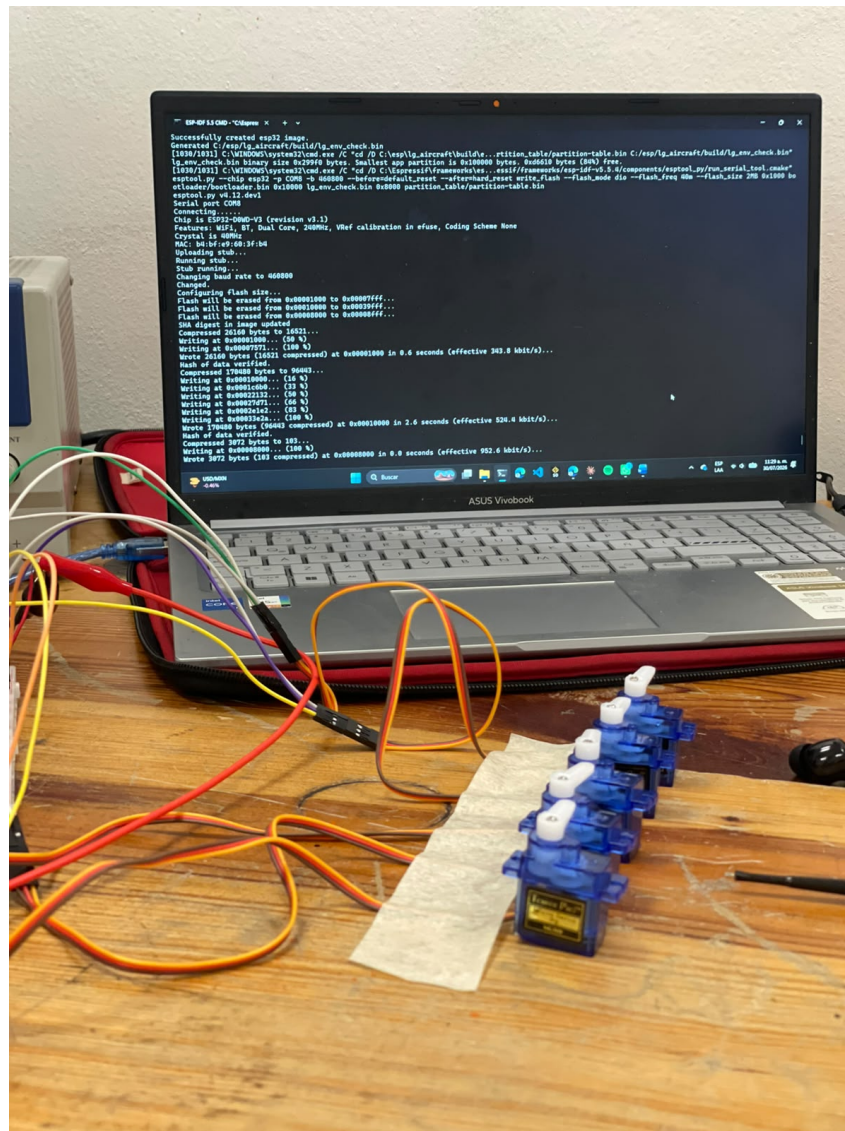


Figure 4: Five servos sweeping in unison from one LEDC timer.

7 Main Activity

7.1. Description

Assemble the complete flight-control system. The generated model reads `mode` and `dir`, computes the four surface angles, the flight phase, the panel LEDs, and its `emergency` output. Five tasks surround it: `sensor` samples the pots, `model` steps the model on a strict 100 ms cycle and distributes its outputs, `actuator` drives the five servos, `safety` kicks the watchdog, and `telemetry` logs the aircraft state. Moving the pots flies the surfaces; disconnecting a pot makes the model declare `emergency`, drive the surfaces to neutral, and flash red.

7.2. Functional Specifications

- **Spec 1:** The full model is reused unchanged; the glue matches the generated step signature exactly (confirm in the header).
- **Spec 2:** `sensor_task` samples `mode` and `dir` (ADC1) and publishes them; `model_task` steps the model every 100 ms with `vTaskDelayUntil`.
- **Spec 3:** `actuator_task` writes the four surface angles and the gear angle to five LEDC channels; all five come directly from the model outputs (the gear is a model int32, 90° down / 0° up).
- **Spec 4:** `safety_task` (highest priority) kicks the Task Watchdog; the model's own emergency output forces the surfaces to neutral and flashes the red LED.
- **Spec 5:** `telemetry_task` logs one line per state change: phase, surface angles, and emergency status.

7.3. Activity Steps

1. Generate and confirm the model interface.

Generate the complete model and copy its `scade_gen` into the project. Open the header and write down the exact step name and the order of its arguments — this is the contract the glue must match. The listing below uses representative names; *adapt every one* to your header.

2. Write the integration glue.

Replace `main.c`. Servo/ADC helpers are the Worked Example's; only the tasks change.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "freertos/semphr.h"
#include "driver/gpio.h"
#include "driver/ledc.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_task_wdt.h"
#include "esp_log.h"
#include "operator0_module0.h"    // generated full model - confirm name

#define LED_RED 25
#define LED_GRN 26
#define LED_WHT 33
static const char *TAG = "AC";
static const int SERVO_GPIO[5] = { 13, 12, 14, 27, 4 }; // ale L/R, rudder, elev, gear

typedef struct { int s[5]; } cmd_t;    // five servo angles
typedef struct { int mode, dir; } in_t; // two pilot inputs

static QueueHandle_t q_act;
static SemaphoreHandle_t logmux;
static volatile in_t pilot = {0,0};
static volatile int g_phase = 0;
static volatile bool g_emerg = false;

// --- servo helpers (Worked Example) ---
static void servos_init(void){ ledc_timer_config_t t={.speed_mode=LEDC_LOW_SPEED_MODE,
    .timer_num=LEDC_TIMER_0,.duty_resolution=LEDC_TIMER_16_BIT,.freq_hz=50,
    .clk_cfg=LEDC_AUTO_CLK}; ledc_timer_config(&t);
    for(int i=0;i<5;i++){ ledc_channel_config_t c={.gpio_num=SERVO_GPIO[i],
        .speed_mode=LEDC_LOW_SPEED_MODE,.channel=i,.timer_sel=LEDC_TIMER_0,
```

```

        .duty=0,.hpoint=0}; ledc_channel_config(&c);} }
static void servo_write(int ch,int deg){ if(deg<0){deg=0;} if(deg>180){deg=180;}
    int us=500+(2500-500)*deg/180; uint32_t d=(uint32_t)((uint64_t)us*65535/20000);
    ledc_set_duty(LEDC_LOW_SPEED_MODE,ch,d); ledc_update_duty(LEDC_LOW_SPEED_MODE,ch); }

// --- sensor: two pots -> pilot inputs ---
static void sensor_task(void *arg){
    adc_oneshot_unit_handle_t adc;
    adc_oneshot_unit_init_cfg_t u={.unit_id=ADC_UNIT_1}; adc_oneshot_new_unit(&u,&adc);
    adc_oneshot_chan_cfg_t ch={.atten=ADC_ATTEN_DB_12,.bitwidth=ADC_BITWIDTH_DEFAULT};
    adc_oneshot_config_channel(adc,ADC_CHANNEL_6,&ch); // GPIO34 mode
    adc_oneshot_config_channel(adc,ADC_CHANNEL_7,&ch); // GPIO35 dir
    while(1){ int m,d;
        adc_oneshot_read(adc,ADC_CHANNEL_6,&m);
        adc_oneshot_read(adc,ADC_CHANNEL_7,&d);
        pilot.mode=m; pilot.dir=d;
        vTaskDelay(pdMS_TO_TICKS(50)); }
}

// --- control: the full model on a strict 100 ms cycle ---
static void model_task(void *arg){
    gpio_set_direction(LED_RED,GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_GRN,GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_WHT,GPIO_MODE_OUTPUT);
    outC_operator0_module0 ctx; operator0_reset_module0(&ctx);
    esp_task_wdt_add(NULL);
    TickType_t last=xTaskGetTickCount(); bool blink=false;
    while(1){
        esp_task_wdt_reset();
        // model outputs (declare all; use what YOUR header defines)
        swan_bool takeoff,ground,cruise,landing,red,green,white,emergency,safe;
        swan_int32 ail_l,ail_r,rudder,elev,gear,dirf,cont;
        operator0_module0(pilot.mode, pilot.dir,
            &takeoff,&ground,&cruise,&landing,
            &ail_l,&ail_r,&rudder,&elev,&gear, // gear is a model output
            &red,&green,&white,&dirf,&emergency,&cont,
            &safe,&ctx);
        cmd_t cmd = { .s = { (int)ail_l,(int)ail_r,(int)rudder,(int)elev,(int)gear } };
        xQueueSend(q_act,&cmd,0);
        // panel LEDs (model drives them; red flashes in emergency)
        gpio_set_level(LED_GRN, green);
        gpio_set_level(LED_WHT, white);
        if(emergency){ blink=!blink; gpio_set_level(LED_RED,blink); }
        else { gpio_set_level(LED_RED, red); }
        g_phase = ground?0:takeoff?1:cruise?2:landing?3:0;
        g_emerg = emergency;
        vTaskDelayUntil(&last,pdMS_TO_TICKS(100));
    }
}

// --- actuator: five servos ---
static void actuator_task(void *arg){
    servos_init(); cmd_t c;
    while(1) if(xQueueReceive(q_act,&c,portMAX_DELAY)==pdTRUE)
        for(int i=0;i<5;i++) servo_write(i,c.s[i]);
}

// --- telemetry: log the aircraft state on change ---
static void telemetry_task(void *arg){
    const char *ph[]={"GROUND","TAKEOFF","CRUISE","LANDING"};
    int last=-1; bool le=false;
    while(1){
        if(g_phase!=last || g_emerg!=le){
            xSemaphoreTake(logmux,portMAX_DELAY);
            ESP_LOGW(TAG,"phase=%s emergency=%d",ph[g_phase],(int)g_emerg);
            xSemaphoreGive(logmux); last=g_phase; le=g_emerg;
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void app_main(void){
    q_act = xQueueCreate(4,sizeof(cmd_t));
}

```

```

logmux = xSemaphoreCreateMutex();
xTaskCreate(model_task, "model", 4096, NULL, 4, NULL);
xTaskCreate(actuator_task, "act", 3072, NULL, 3, NULL);
xTaskCreate(sensor_task, "sens", 3072, NULL, 3, NULL);
xTaskCreate(telemetry_task, "t1m", 3072, NULL, 2, NULL);
}

```

3. Flash and fly.

idf.py -p COM8 flash monitor. Turn mode to walk the flight phase (LEDs and gear follow); turn dir to deflect the ailerons and rudder. Then *pull the dir pot's wiper wire* to simulate a lost sensor: the model raises **emergency**, the surfaces snap to neutral, and the red LED flashes.

```

W (217421) AC: phase=GROUND emergency=1
W (219921) AC: phase=GROUND emergency=0
W (220821) AC: phase=GROUND emergency=1
W (221321) AC: phase=GROUND emergency=0
W (222121) AC: phase=GROUND emergency=1
W (222821) AC: phase=GROUND emergency=0
W (224121) AC: phase=GROUND emergency=1
W (225521) AC: phase=GROUND emergency=0
W (238721) AC: phase=GROUND emergency=1
W (239221) AC: phase=GROUND emergency=0
W (572621) AC: phase=GROUND emergency=1
*****573021) AC: phase=GROUND emergency=0
W (574121) AC: phase=TAKEOFF emergency=0
W (575321) AC: phase=GROUND emergency=0
W (575421) AC: phase=GROUND emergency=1
W (576321) AC: phase=GROUND emergency=0
W (583021) AC: phase=TAKEOFF emergency=0
W (583221) AC: phase=CRUISE emergency=0
W (583521) AC: phase=LANDING emergency=0
W (583621) AC: phase=LANDING emergency=1
W (583721) AC: phase=GROUND emergency=1
W (584321) AC: phase=GROUND emergency=0
W (585121) AC: phase=GROUND emergency=1
W (585521) AC: phase=GROUND emergency=0
W (586621) AC: phase=TAKEOFF emergency=0
W (588921) AC: phase=CRUISE emergency=0
W (590621) AC: phase=LANDING emergency=0
W (591321) AC: phase=CRUISE emergency=0
W (591921) AC: phase=TAKEOFF emergency=0
W (593521) AC: phase=GROUND emergency=0
W (599121) AC: phase=TAKEOFF emergency=0
W (603121) AC: phase=GROUND emergency=0
W (604021) AC: phase=TAKEOFF emergency=0
W (604821) AC: phase=GROUND emergency=0
*****862621) AC: phase=GROUND emergency=1
W (862821) AC: phase=GROUND emergency=0
W (864421) AC: phase=TAKEOFF emergency=0
W (866821) AC: phase=CRUISE emergency=0

```

Figure 5: Full system: pots fly the surfaces, phase drives the gear, a pulled input triggers the model's emergency.

8 Expected Results

Expected Results

- **SCADE Simulation:** The model produces the four surface angles and the phase from mode/dir, and raises **emergency** on a lost or jumping input, forcing the surfaces to neutral — all verified before flashing.
- **Console / UART Output:** One line per state change:


```

W (1900) AC: phase=TAKEOFF emergency=0
W (4200) AC: phase=CRUISE emergency=0
W (6100) AC: phase=CRUISE emergency=1
W (8300) AC: phase=GROUND emergency=0

```

- **Physical Behavior:** Turning `mode` steps the phase (LEDs + gear servo follow); turning `dir` deflects the aileron, rudder, and elevator servos. Pulling an input drives all surfaces to neutral (90°) and flashes the red LED — the model's emergency, running live. Under all of this the control cycle stays at 100 ms.

9 Assessment Questionnaire

Answer each question based on your specific implementation. Justify your responses with references to your design decisions.

Q1. [*Comprehension*] List every output your generated model produces and state, for each, whether the glue turns it into a servo command, an LED, a telemetry field, or nothing. Identify which single output makes the whole system fail safe, and trace where the glue acts on it.

Q2. [*Analysis*] Five servos share one 50 Hz LEDC timer across five channels. Explain why they can share the timer, what would happen to the surface timing if the actuator task were given a lower priority than telemetry, and how the queue decouples the servo writes from the control cycle.

Q3. [*Analysis*] The emergency logic lives entirely inside the SCADE model, while in Practice 8 you built the detection in the glue. Compare the two designs: what does having the model detect *and* react give you for verification, and what does it cost in flexibility?

Q4. [*Synthesis*] A reviewer asks for the surface angles to be streamed to a ground station over a real bus, not the console. Describe how you would add a framed UART (Practice 4) or CAN link as another consumer task without touching the control task, and what fields the frame should carry.

Q5. [*Critical*] Identify the two most safety-critical single points of failure in this system as built (consider power, the shared timer, the one control task, the sensor wiring). For each, propose a concrete mitigation and state whether it belongs in the model, the glue, or the hardware.

10 Conclusion

Write a reflective conclusion on the complete system and the course. Trace how one unchanging principle — the model owns the logic, the platform owns the I/O — carried from the first generated operator to this full flight controller. Assess what the RTOS architecture bought over a single loop, why the emergency behaviour belongs in the model, and what you would do next to take this design toward a certifiable product.

11 Bonus Challenge

★ Bonus Challenge

Challenge:

Give the aircraft a ground station. Add a `telemetry_task` that, instead of logging text, frames the full state — phase, four surface angles, gear, emergency — into a compact binary packet (Practice 4) and sends it over UART2, validated on the loopback. Then write a tiny host script (Python + `pyserial`) that decodes the packets and prints a live dashboard of the surfaces. You now have a flight computer *and* a monitoring console.

Hint:

Reuse the framing discipline of Practice 4: a header byte, the payload fields in a fixed order, and a checksum. Keep the packet small and send it every cycle; the host re-syncs on the header, exactly like the bonus receiver you built there.

12 Troubleshooting

This section is populated with real issues encountered during development. If you run into a problem not listed here, document it using the same format below.

Issue: The board resets or servos twitch when several move

Symptom: Brown-out resets, or servos jitter, when more than one surface moves at once.

Cause: Five servos exceed the supply's current; the 5 V rail sags under the combined draw.

Fix: Use a ≥ 2 A 5 V supply on a solid rail with a common ground; verify the rail holds 5 V while several servos move together. This is the defining hardware challenge of the practice.

Issue: Too-few / too-many arguments on the model step

Symptom: The compile fails on `operator0_module0` arguments, or reads garbage from some outputs.

Cause: The glue's call does not match *your* generated signature — the representative names and order in the listing differ from your header.

Fix: Open the generated header, copy the exact argument list, and align the glue variable-for-variable. This is the same header-first discipline from Practice 3.

Issue: A servo does not move / LEDC channel error

Symptom: One channel is dead or configuration fails on a channel index.

Cause: Two channels bound to the same GPIO, a pin used by something else, or more channels than the timer serves.

Fix: Keep channels 0–4 unique on their five GPIOs; do not reuse a servo pin for an LED. All five channels point at `LEDC_TIMER_0`.

Issue: Emergency never triggers when a pot is pulled

Symptom: Disconnecting an input does nothing.

Cause: A floating ADC pin can read mid-scale instead of a clear extreme, so the model's disconnect detector does not fire; or the **emergency** output is not wired to the red LED / neutral command.

Fix: Confirm the model's emergency logic in the simulator first; ensure the glue reads **emergency** and both flashes red and lets the model's neutral surface angles reach the servos.